Assignment No	**13**
Name	**Satish Rathod**
Title	**Application of Linked Lists - Addition of Polynomial & List implementation of Stacks**
Objective	**Polynomial Addition & List Stack**
Roll No	**MCA2649**

1)Addition of Polynomial

Source Code:-

```
//Name:Satish Rathod
//Roll No:MCA2649
//Module 4: Linked Lists
// Application of Linked Lists - Addition of Polynomial


import java.util.*;
//Node Template
class PNode
{
        int coeff;
        int exp;
        PNode next;


        PNode(int c,int e)
```

```
        {
                this.coeff = c;
                this.exp= e;
                this.next = null;
        }
}//end of pnode

//List Template
class Polynomial
{
        PNode head;

        //Insert a node in order of exponent
        void Insert(int coeff,int exp)
        {
                PNode t = new PNode(coeff,exp);

                //If the list is empty or exponent is greater than head
                if(head==null || exp>head.exp)
                {
                        t.next = head;
                        head = t;
                        return;
                }
```

```
        //Traversal
        PNode temp = head;
        PNode prev = null;

        while(temp!=null && temp.exp>exp)
        {
                prev = temp;
                temp = temp.next;
        }
        prev.next = t;
        t.next = temp;


}//end of Insert

//Display the polynomial
void Display()
{
        if(head == null)
        {
                System.out.println("Empty Polynomial");
                return;
        }

        PNode temp = head;
```

```
        while(temp!=null)
        {
                System.out.print(temp.coeff +"x^"+temp.exp);
                if(temp.next!=null)
                {
                        System.out.print(" + ");
                }
                temp = temp.next;
        }
}//end of display

static Polynomial addPolynomial(Polynomial p1, Polynomial p2)
{
        Polynomial result = new Polynomial();

        PNode a = p1.head;
        PNode b = p2.head;

        while(a!=null && b!=null)
        {
                if(a.exp == b.exp)
                {
                        int sum = a.coeff + b.coeff;
                        if(sum!=0)
                        {
```

```
                    result.Insert(sum,a.exp);
                }
                a = a.next;
                b = b.next;
        }
        else if(a.exp > b.exp)
        {
                result.Insert(a.coeff,a.exp);
                a = a.next;
        }
        else
        {
                result.Insert(b.coeff,b.exp);
                b = b.next;
        }
}//end of while

//add any other remaonon terms
while(a!=null)
{
        result.Insert(a.coeff,a.exp);
        a= a.next;
}
while(b!=null)
{
```

```
                result.Insert(b.coeff,b.exp);
                b= b.next;
            }
            return result;

        }//end of addPolynomial
}//end of Polynomial

class AddP
{
        public static void main(String args[])
        {
            Scanner sc = new Scanner(System.in);

            Polynomial p1 = new Polynomial();
            Polynomial p2 = new Polynomial();

            System.out.println("Enter the number of terms in the first
Polynomial. ");
            int n1 = sc.nextInt();
            System.out.println("Enter each terms as: coefficient exponent ");

            for(int i=0;i<n1;i++)
            {
                int c = sc.nextInt();
                int e = sc.nextInt();
```

```
                p1.Insert(c,e);
            }

            System.out.println("Enter the number of terms in the first
Polynomial. ");
            int n2 = sc.nextInt();
            System.out.println("Enter each terms as: coefficient exponent ");

            for(int i=0;i<n2;i++)
            {
                int c = sc.nextInt();
                int e = sc.nextInt();
                p2.Insert(c,e);
            }

            System.out.print("Polynomial 1: ");
            p1.Display();

            System.out.println("\nPolynomial 2: ");
            p2.Display();

            Polynomial res = Polynomial.addPolynomial(p1,p2);

            System.out.print("\nResult: ");
            res.Display();
        }//end of the main
```

}

Output:-

1)

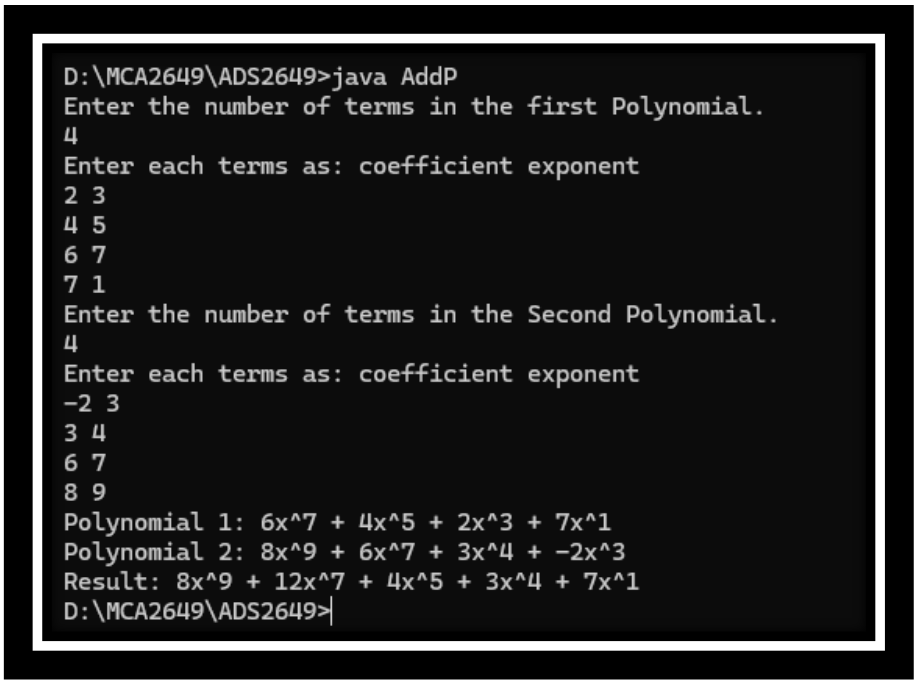
```
D:\MCA2649\ADS2649>java AddP
Enter the number of terms in the first Polynomial.
4
Enter each terms as: coefficient exponent
2 4
-3 2
5 1
3 0
Enter the number of terms in the Second Polynomial.
4
Enter each terms as: coefficient exponent
6 3
3 2
4 1
-1 0
Polynomial 1: 2x^4 + -3x^2 + 5x^1 + 3x^0
Polynomial 2: 6x^3 + 3x^2 + 4x^1 + -1x^0
Result: 2x^4 + 6x^3 + 9x^1 + 2x^0
D:\MCA2649\ADS2649>
```

2)

```
D:\MCA2649\ADS2649>javac AddP.java

D:\MCA2649\ADS2649>java AddP
Enter the number of terms in the first Polynomial.
4
Enter each terms as: coefficient exponent
2 3
4 5
6 7
4 3
Enter the number of terms in the first Polynomial.
2
Enter each terms as: coefficient exponent
4 5
6 3
Polynomial 1: 6x^7 + 4x^5 + 4x^3 + 2x^3
Polynomial 2: 4x^5 + 6x^3
Result: 6x^7 + 8x^5 + 2x^3 + 10x^3
D:\MCA2649\ADS2649>
```

3)



2) - List Implementation of a Stack

Source Code:-

```
//Name:Satish Rathod
//Roll No:MCA2649
//Module 4: Linked Lists
// Application of Linked Lists - List Implementation of a Stack

import java.util.*;

class SNode
{
```

```
        int data;
        SNode next;

        public SNode(int d)
        {
                this.data= d;
                this.next = null;
        }
}//end of SNode

//List Template for Stack
class SList
{
        SNode tos;

        public SList()
        {
                tos = null;
        }

        //push operation
        public void Push(int d)
        {
                SNode x = new SNode(d);
```

```
        if(tos == null)
        {
                tos = x;
        }
        else
        {
                x.next = tos;
                tos = x;
        }

}//end of the push

//POP operation
public void Pop()
{
        if(tos == null)
        {
                System.out.println("Stack Underflow!");
        }
        else
        {
                int t = tos.data;
                tos = tos.next;

                System.out.println("Popped element: " + t);
```

```
        }
}//end of the pop

//Display
public void Display()
{
        if(tos == null)
        {
                System.out.println("Stack Underflow!");
        }
        else
        {
                SNode temp = tos;

                System.out.println("Stack Contains!");
                while(temp!=null)
                {
                        System.out.println(temp.data);
                        temp = temp.next;
                }
        }
}//end of the Display

//Peek
public void Peek()
```

```
        {
                if(tos == null)
                {
                        System.out.println("Stack Underflow!");
                }
                else
                {
                        System.out.println("Element at the TOS: " + tos.data);

                }
        }//end of the Peek
}//end of SList

class LStack
{
        public static void main(String args[])
        {
                SList s = new SList();

                for(int i=1; i<=5; i++)
                {
                        s.Push(i*10);
                        s.Display();
                }
```

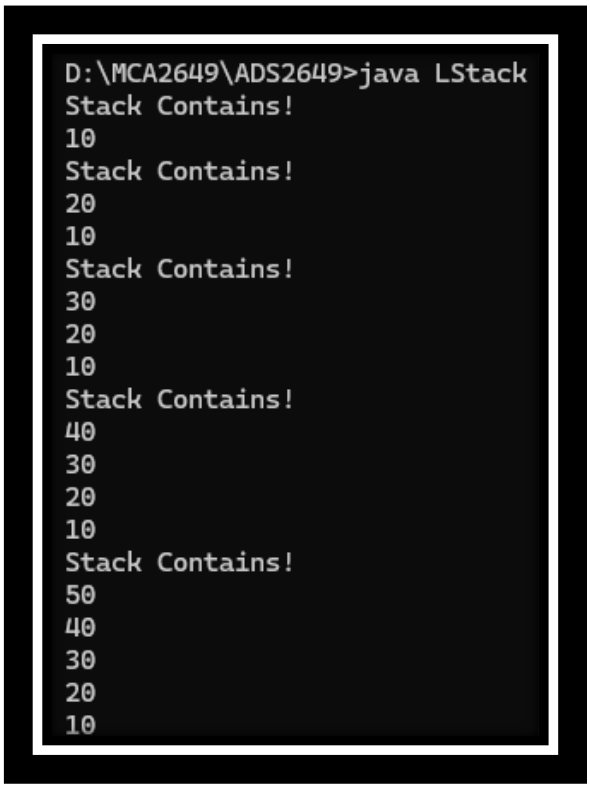
```
            s.Peek();


            for(int i=1; i<=5 ; i++)
            {
                  s.Pop();
                  s.Display();
            }
            s.Pop();
      }//end of the main function

}//end of the main class
```

Output:-

1)



2)

```
Element at the TOS: 50
```

3)

```
Popped element: 50
Stack Contains!
40
30
20
10
Popped element: 40
Stack Contains!
30
20
10
Popped element: 30
Stack Contains!
20
10
Popped element: 20
Stack Contains!
10
Popped element: 10
Stack Underflow!
Stack Underflow!
```